EvoCracker

AI RESEARCH PROJECT

Phase 4 of 5 — AI Evolution Proof & Algorithm Comparative Analysis

PHASE 4

AI Evolution Proof

Algorithm comparative analysis & dynamic evidence of real-time adaptation

Deliverable Summary

- Replaced static accuracy framing with measurable live evolution evidence.
- Connected enemy runtime telemetry directly to dashboard proof views.
- Added comparative algorithm runtime metrics for transparent benchmarking.
- Solved algorithm diversity collapse by assigning a default algorithm per enemy type — all 8 algorithms are guaranteed present in the dashboard at all times.

Verification Checklist

✓ Evolution evidence uses live runtime values, not synthetic filler data.
✓ Comparative pathfinding metrics are measurable in active sessions.
✓ Demonstration flow is reproducible for evaluator review.
✓ All 8 search algorithms are observable from round one via default enemy-algorithm mapping.

1. Replacing ML Accuracy Scores with Evolution Graphs

In a standard ML environment, accuracy scores (F1, precision) prove intelligence. In EvoCracker, intelligence is proven through Emergent Fitness and Genetic Shifts, visualized live in the AI Analytics Panel (accessible via the ` key).

Evolution Dashboard

- Generation Fitness Charts — A bar chart tracking average, median, and max fitness across dungeon floors. A steady upward trend proves the Genetic Algorithm is successfully weeding out weak enemies.
- Gene Averages — Visualizes specific genetic traits (Speed, Vision, Aggression, Caution, Pack Tendency, Ambush, Patrol) mutating over time.
- Iteration Proof — Stats are extracted 100% authentically from living enemies. The Strength Index Delta shows exactly how much stronger the current generation's genome is versus the baseline.

Example

If a player rushes, the population's Speed and Aggression graphs rise over generations as slow enemies are killed off. If a player hides often, Vision and Persistence graphs spike as enemies adapt to search longer.

2. Authentic Telemetry Extraction

Metrics are pulled directly from living AI agents via getAnalyticsSnapshot() in EnemyBase.ts twice a second. To ensure graphs are not filler data, the system pulls:

- Exact physical states — current health, speed.
- Genetic weights — 0.0 to 1.0 trait values.
- Living performance data — damage dealt, unique tiles explored, time stuck.

3. Search Algorithm Comparative Analysis

Each enemy type has a hardcoded default algorithm assigned at spawn. This was introduced because free genetic evolution caused most of the population to converge on the same 1–2 high-performing algorithms after a few generations, collapsing algorithm diversity in the dashboard. With the default mapping in place, all 8 algorithms are guaranteed to appear from the first spawn of every run.

Implementation Note

Algorithm selection is fully controlled by a hardcoded per-enemy-type default — not by genetic evolution. The algorithmWeights gene and its mutation logic were implemented and tested, but caused the population to converge onto 1–2 algorithms within a few generations. The default mapping replaced it entirely. The GA continues to evolve all other behavioral traits normally.

Metric	What It Measures
Average Compute Time (ms)	Real-time JavaScript execution cost of calculating a route to the player.
Average Nodes Expanded	Memory and search-space efficiency. A* predictably expands fewer nodes than BFS in open spaces.

By tracking nodesExpanded and pathComputeTimeMs across all 8 guaranteed-present algorithms, we can directly compare how different search strategies perform on the same dungeon grid — proving that A* and Greedy BFS are faster and more node-efficient than DFS or Hill Climbing, with concrete numbers to back it up.

4. How to Demonstrate

Recommended Evaluator Flow:

1	Start the game and complete a floor.
2	Open the AI panel with the backtick (`) key.
3	Open the Player tab to show the AI has classified the player's playstyle based on raw keystrokes, movement samples, and dominant zones.
4	Open the Evolution tab to display the fitness chart climbing over multiple generations, proving the enemies are learning.
5	Open the Pathfinding tab — all 8 algorithms are visible immediately because each enemy type has a hardcoded default algorithm. Point out the distinct Compute Time and Nodes Expanded values per algorithm (e.g. A* vs. DFS vs. Hill Climbing).
6	Observe enemy movement styles in gameplay — DFS Stalkers dive deep corridors, A* Hunters take optimal routes, Hill Climbing Ambushers creep locally. Each default algorithm produces a visibly different chase pattern.